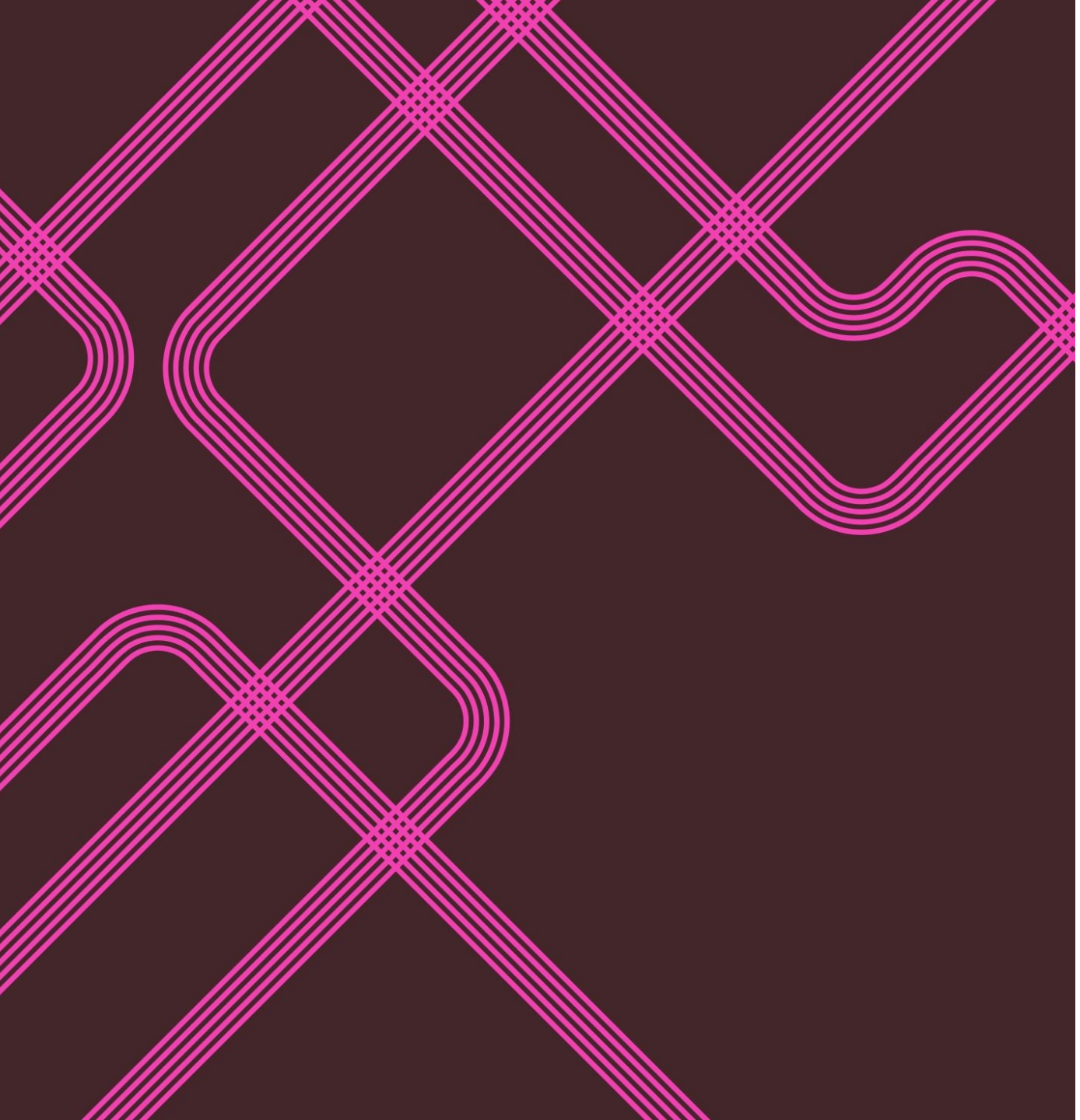


Tema 67 (I)

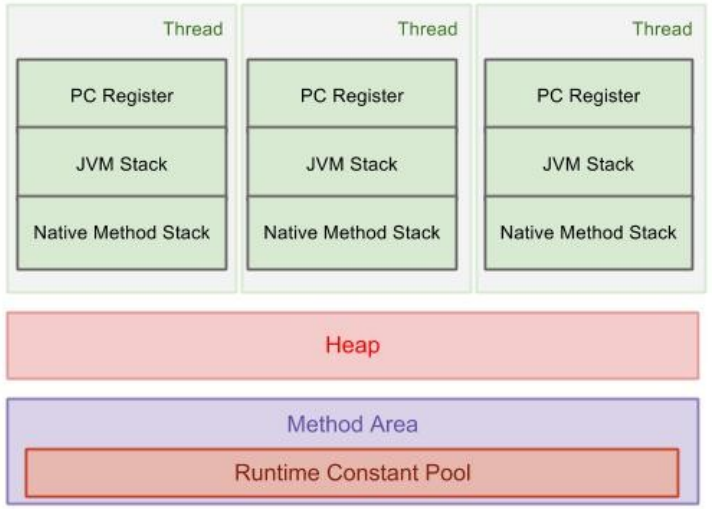
Entorno de desarrollo JAVA

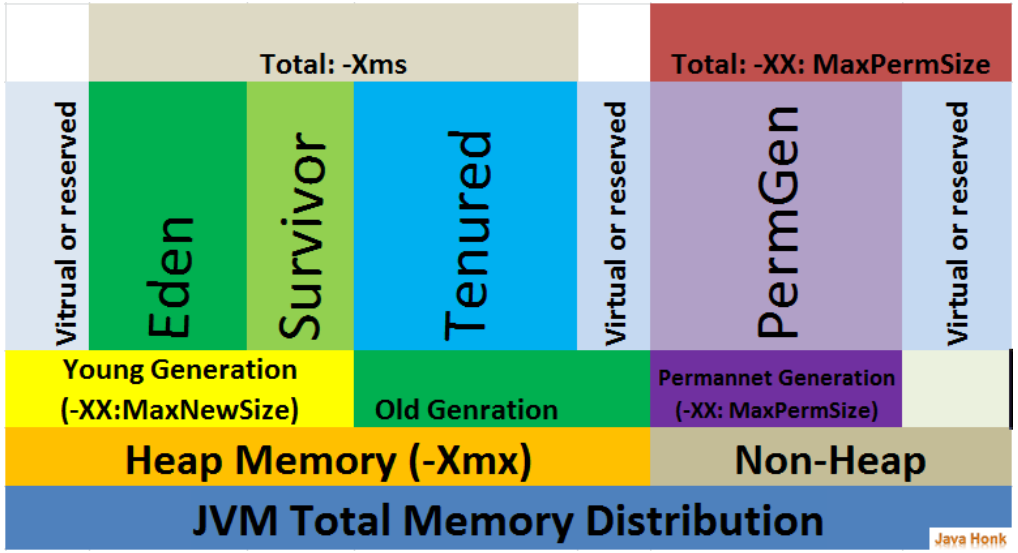
Introducción a Java

- Fue desarrollado por Sun Microsystems, lanzado en 1995.
- Se caracteriza por su filosofía de Write Once, Run Anywhere (WORA), permite ejecutar en cualquier sistema gracias a la JVM.
- Ventajas sobre C/C++:
  - Sin punteros directos.
  - Gestión automática de memoria mediante el recolector de basura.
- Desde el principio tuvo como propósito ser una plataforma y lenguaje orientado a portabilidad, seguridad y robustez.
- Ha estado en evolución constante desde Java 1.0 a Java 21, con hitos técnicos y cambios en el ecosistema.

Gestión automática de memoria en Java (GC)

- Heap (objetos)
  - Young Gen: Eden + Survivor (objetos jóvenes).
  - Old Gen: objetos de larga vida.
  - Metaspace: metadatos de clases.
- Ciclo de vida
  1. Asignación en Eden (rápida).
  2. Copia entre Survivor si siguen vivos.
  3. Promoción a Old tras varias colecciones.





Aparición de Java y primeras versiones (1.0 – 1.1)

- Orígenes: Proyecto Green (1991, Sun Microsystems) → Oak → renombrado Java (1994).
- Lanzamiento: 1995, con foco en applets para aplicaciones web interactivas.
- Java 1.0 (1996)
  - APIs y clases básicas.
  - Biblioteca gráfica AWT.
  - Bytecode y primera JVM.
  - Portabilidad independiente del SO (WORA).
- Java 1.1 (1997)
  - Ampliación API estándar (Reader/Writer, manejo de fechas, Resource Bundles).
  - JavaBeans, JDBC (acceso a BBDD).
  - Clases internas (inner classes) y RMI.

JavaBeans en java 1.1

- Componente reutilizable de software pensado para herramientas visuales.
- Reglas:
  - Constructor sin argumentos.
  - Propiedades con getters y setters.
  - Implementa Serializable.
  - Usados en entornos como VisualAge, JBuilder, NetBeans.
  - Configuración en tiempo de diseño (drag & drop en IDE).

```
public class PersonaBean implements Serializable {
    private String nombre;
    public String getNombre() { return nombre; }
    public void setNombre(String n) { this.nombre = n;
}
}
```

JDBC (Java Database Connectivity)

- API estándar para conectar Java con bases de datos relacionales.
- Sustituyó librerías propietarias → unificación del acceso.
- Arquitectura básica
  - DriverManager – gestiona drivers JDBC.
  - Connection – establece la conexión
  - .Statement / PreparedStatement – ejecuta SQL.
  - ResultSet – recorre resultados.

Clases Internas (Inner Classes)

- Permiten definir una clase dentro de otra para encapsular lógica y mejorar la legibilidad.
- Tipos
  - No estáticas → acceden a miembros de la clase externa.
  - Estáticas → como clases independientes, pero agrupadas.
  - Anónimas → implementación rápida de interfaces o clases abstractas (muy usadas antes de lambdas).

```
class Outer {
    class Inner {
        void print() { System.out.println("Hola"); }
    }
}
```

RMI (Remote Method Invocation)

- API para invocar métodos en objetos remotos como si fueran locales.
- Facilita la programación distribuida 100% Java.
- Cómo funciona
  - Stub/Skeleton: proxies generados automáticamente.
  - Comunicación sobre TCP/IP.
  - Serialización de objetos para paso de parámetros y retorno de valores.

Java 1.2 – 1.4 (I)

- 1998: Lanzamiento de Java 1.2 (Java 2), gran salto evolutivo.
- Segmentación de la plataforma:
  - Java SE – Escritorio
  - Java EE – Empresarial/servidor
  - Java ME – Móviles/embebidos
- Java 1.2 (1998)
  - Framework de Colecciones.
  - Swing (GUIs avanzadas) y Java 2D.
  - Seguridad mejorada (firmas digitales, controles de acceso).
  - Internacionalización y mejoras JDBC.

Java 1.2 – 1.4 (II)

- Java 1.3 (2000)
  - JVM HotSpot con JIT.
  - API JNDI.
  - Mejoras en RMI, Swing y despliegue (caché de clases, descarga selectiva de JARs).
- Java 1.4 (2002)
  - Assertions y regex.
  - SSL/TLS, Kerberos, NIO, logging.
  - Procesamiento XML integrado.
  - Java Web Start oficial.
  - Compatibilidad con 64 bits.

Assertions (assert)

- Introducción de la palabra clave assert en el lenguaje.
- Sirve para comprobar que ciertas condiciones lógicas se cumplen en tiempo de ejecución.
- Pensadas como herramienta de depuración.

```
int edad = -5;
assert edad >= 0 : "La edad no puede ser negativa";
```

Expresiones regulares (Regex)

- Antes de Java 1.4, si se quería regex había que usar librerías externas (ej. Apache ORO).
- En 1.4 se introdujo java.util.regex → soporte oficial de expresiones regulares.

```
import java.util.regex.*;

public class RegexEjemplo {
    public static void main(String[] args) {
        String texto = "Mi correo es usuario@mail.com";
        Pattern p = Pattern.compile("\\w+@\\w+\\.\\w+");
        Matcher m = p.matcher(texto);

        if (m.find()) {
            System.out.println("Encontrado: " + m.group());
        }
    }
}
```

SSL/TLS (JSSE integrado)

- HTTPS y sockets seguros en la API estándar.
- Uso de certificados y cifrado en aplicaciones Java.

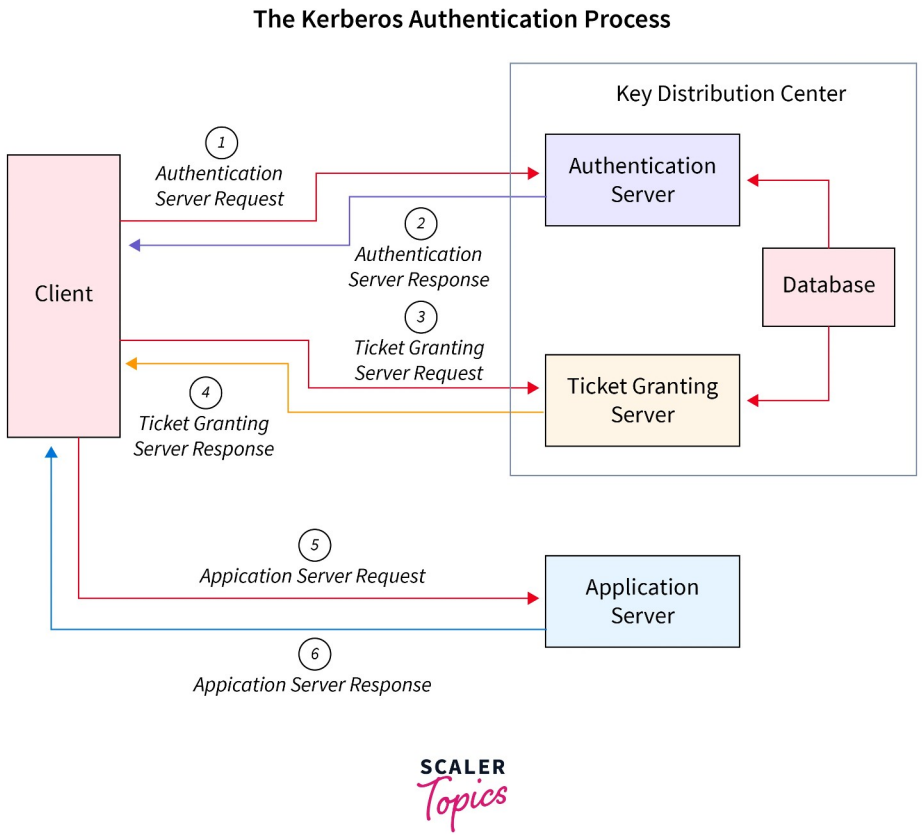
```
import javax.net.ssl.HttpsURLConnection;
import java.net.URL;

public class HTTPSDemo {
    public static void main(String[] args) throws Exception {
        URL url = new URL("https://example.com");
        HttpsURLConnection con = (HttpsURLConnection) url.openConnection();

        System.out.println("Cifrado usado: " + con.getCipherSuite());
    }
}
```

Kerberos

- Autenticación integrada en javax.security.auth.kerberos.
- Compatible con JAAS y entornos corporativos (SSO).



NIO (New I/O)

- Nuevo paquete java.nio.
- Buffers + Channels → E/S más rápida.
- Non-blocking I/O para servidores concurrentes.

```
FileChannel channel = new FileInputStream("ejemplo.txt").getChannel();
ByteBuffer buffer = ByteBuffer.allocate(64);

while (channel.read(buffer) != -1) {
    buffer.flip();                  // modo lectura
    while (buffer.hasRemaining())
        System.out.print((char) buffer.get());
    buffer.clear();                 // modo escritura
}
```

Sistema de logging estándar

- Sistema estándar de logs.
- Clases: Logger, Handler, Formatter.
- Niveles: INFO, WARNING, SEVERE...

```
import java.util.logging.*;

public class LogEjemplo {
    private static final Logger logger = Logger.getLogger("MiApp");

    public static void main(String[] args) {
        logger.info("Iniciando aplicación...");
        logger.warning("Advertencia: uso alto de memoria");
        logger.severe("Error crítico");
    }
}
```

JNDI (Java Naming and Directory Interface)

- API estándar de Java para localizar recursos por nombre (sin codificar direcciones físicas).
- Usada para :
  - Lookup de DataSources JDBC (BBDD).
  - JavaMail sessions.
  - Integración con LDAP, DNS, RMI registry.
  - Configuración centralizada en el servidor de aplicaciones.
  - JMS Queues/Topics.

Procesamiento XML integrado

- Con 1.4 se incorporó oficialmente:
  - JAXP (Java API for XML Processing) dentro del JDK.
  - Implementaciones DOM y SAX ya venían integradas (basadas en Xerces).

```
import javax.xml.parsers.*;
import org.w3c.dom.*;

File xml = new File("ejemplo.xml");
DocumentBuilder db =
DocumentBuilderFactory.newInstance().newDocumentBuilder();
Document doc = db.parse(xml);

System.out.println("Elemento raíz: " +
doc.getDocumentElement().getNodeName());
```

Java Web Start (oficial en 1.4)

- Tecnología para ejecutar aplicaciones Java desde la web con un clic (sin instalar manualmente JARs).
- El navegador descargaba un fichero .jnlp (Java Network Launch Protocol), que contenía:
  - Nombre de la app.
  - Recursos necesarios (JARs).
  - Permisos de seguridad.

Java 5

- Java 5.0 (1.5, 2004) – Tiger
  - Genéricos – comprobación de tipos en compilación.
  - Anotaciones (annotations) – metadatos para frameworks y reflexión.
  - Enumeraciones, for-each, varargs, autoboxing/unboxing.
  - java.util.concurrent – estructuras concurrentes y thread pools.
  - JDBC 3.0 y otras utilidades estándar.

Genéricos en Java 5

- Se puede declarar el tipo de objetos que contendrá la colección (o cualquier clase genérica).

```
class Caja<T> {
    private T valor;
    public void set(T v) { valor = v; }
    public T get() { return valor; }
}

// Uso
Caja<Integer> c1 = new Caja<>();
c1.set(100);
int x = c1.get();  // autounboxing

Caja<String> c2 = new Caja<>();
c2.set("texto");
String y = c2.get();
```

Java 5 – Anotaciones

- Metadatos en el código que guían al compilador, frameworks o herramientas.

```
@Override     // comprueba que sobrescribe
public String toString() { return "Ejemplo"; }

@Deprecated   // marca métodos obsoletos
public void viejo() { }

@SuppressWarnings("unchecked") // oculta warnings
List lista = new ArrayList();
```

Uso en frameworks modernos:

- Spring → @Autowired, @Service
- JPA → @Entity, @Id
- JUnit → @Test

Novedades de sintaxis en Java 5 (I):

- Enumeraciones (enum)
  - Antes de Java 5 → se usaban constantes int o String para representar valores fijos.
  - Ahora → enum es un tipo seguro con valores predefinidos.

```
enum Dia { LUNES, MARTES, MIERCOLES }

Dia d = Dia.LUNES;
```

Novedades de sintaxis en Java 5 (II):

- For-each loop
  - Sintaxis más simple para recorrer colecciones o arrays.

```
for (String nombre : listaNombres) {
    System.out.println(nombre);
}
```

Novedades de sintaxis en Java 5 (III):

- Varargs (Argumentos variables)
  - Permiten pasar un número variable de argumentos a un método.

```
public void sumar(int... numeros) {
    int total = 0;
    for (int n : numeros) total += n;
    System.out.println(total);
}

// Uso:
sumar(1,2,3);   // 6
sumar(10,20);   // 30
```

Novedades de sintaxis en Java 5 (IV):

- Autoboxing / Unboxing
  - Conversión automática entre tipos primitivos y sus wrappers (int ↔ Integer, double ↔ Double).

```
List<Integer> lista = new ArrayList<>();
lista.add(5);         // autoboxing int → Integer
int n = lista.get(0); // unboxing Integer → int
```

Java 5 – java.util.concurrent

- Estructuras concurrentes
  - Colas seguras para hilos (BlockingQueue)

```
BlockingQueue<String> cola = new ArrayBlockingQueue<>(10);
cola.put("tarea");        // productor
String t = cola.take();  // consumidor
```

- Colecciones concurrentes
  - ConcurrentHashMap, CopyOnWriteArrayList → acceso seguro sin bloquear todo el objeto.

- Thread Pools (Ejecutores)
  - ExecutorService → gestiona hilos automáticamente.

Java 6

- Java 6 (1.6, 2006)
  - Servicios web integrados: JAX-WS (SOAP), JAXB (XML binding).
  - API de compilación en tiempo de ejecución (JSR-199).
  - Mejoras en Swing (SwingWorker), scripting embebido (Rhino JS).
  - Herramientas de monitoring y diagnóstico (JVisualVM).
  - Optimizaciones de JVM y rendimiento.

JAX-WS (SOAP Web Services)

- Sustituye a JAX-RPC.
- Permite exponer y consumir servicios SOAP desde el JDK.

```
@WebService
public class HolaService {
   @WebMethod
   public String saludar(String n) { return "Hola " + n; }
}
```

JAXB (XML Binding)

- Convierte objetos Java ↔ XML automáticamente.
- Marshaller y Unmarshaller facilitan serializar y leer XML sin DOM/SAX manual.

```
@XmlRootElement
class Persona { String nombre; int edad; }
```

Java 6 – JSR-199 (Java Compiler API)

- Acceso programático al compilador de Java.
- Permite compilar código en tiempo de ejecución sin invocar javac externo.
- Base de IDEs y frameworks dinámicos.

```
import javax.tools.*;

JavaCompiler compiler = ToolProvider.getSystemJavaCompiler();
int res = compiler.run(null, null, null, "MiClase.java");

System.out.println(res == 0 ? "Compilación exitosa" : "Error al
compilar");
```

Scripting embebido (Rhino JS)

- JSR-223 → API estándar para ejecutar lenguajes de scripting en Java.
- Motor incluido: Rhino (JavaScript).
- Permite ejecutar código dinámico dentro de una app Java.

```
import javax.script.*;

ScriptEngine engine =
   new ScriptEngineManager().getEngineByName("JavaScript");

engine.eval("print('Hola desde JavaScript en Java!');");
engine.put("x", 5); engine.put("y", 10);
engine.eval("print('Suma = ' + (x + y));");
```

Herramientas de Monitoring (JVisualVM)

- Incluido con JDK (desde 6u7).
- Analiza y diagnostica aplicaciones Java en tiempo real.
- Unifica herramientas (jconsole, jmap, jstack, jstat).
- Entre sus funciones principales destacan:
  - Analisis uso de CPU y memoria (incl. GC).
  - Heap dump → inspección de objetos en memoria.
  - Thread dump → detectar bloqueos/deadlocks.
  - Profiling → encontrar cuellos de botella.
  - Conexión a procesos locales y remotos.

Era Oracle: Java 7 y Java 8 (I)

- Java 7 (2011) – Proyecto Coin
  - Strings en switch.
  - Multi-catch (varias excepciones en un solo bloque).
  - Operador diamante <> en genéricos.
  - try-with-resources para cierre automático de recursos.
  - G1 Garbage Collector (grandes montones).
  - invokedynamic para soporte de otros lenguajes en la JVM.

Cambios switches

- Antes de Java 7 el switch solo funcionaba con:
  - Tipos primitivos: int, char, byte, short.
  - Enum (desde Java 5).
- Con java 7 se pasa a soportar String en switch

```
switch (color) {
    case "rojo":
        System.out.println("Color cálido");
        break;
    case "azul":
        System.out.println("Color frío");
        break;
default:
        System.out.println("Color desconocido");
}
```

Java 7 – Multi-catch

- Antes de Java 7 : Cada excepción distinta requería su propio bloque catch, aunque el manejo fuera igual

```
try {
    metodo();
} catch (IOException e) {
    e.printStackTrace();
} catch (SQLException e) {
    e.printStackTrace();
}
```

- Con Java 7: Se pueden capturar varias excepciones en un mismo bloque usando | (pipe):

```
try {
    metodo();
} catch (IOException |
SQLException e) {
    e.printStackTrace();
}
```

Operador diamante <>

- Antes de Java 7 : Al usar genéricos había que repetir el tipo dos veces

```
List<String> lista = new
ArrayList<String>();
```

- Con Java 7: Se introduce el operador diamante (<>) → el compilador infiere el tipo del contexto:

```
List<String> lista = new
ArrayList<>();
```

Try-with-resources

- Los recursos se cierran automáticamente y no es necesario un finally:

```
try (BufferedReader br = new BufferedReader(new
FileReader("archivo.txt"))) {
   String linea = br.readLine();
   System.out.println(linea);
} catch (IOException e) {
    e.printStackTrace();
}
```

Java 7 – G1 Garbage Collector

- G1 (Garbage First): nuevo GC para heaps grandes (multi-GB).
- Introducido en Java 7 (experimental), por defecto desde Java 9.
- Cómo funciona:
  - Divide el heap en regiones en lugar de áreas fijas (joven/viejo).
  - Recolecta primero las regiones con más basura.
  - Paralelo y concurrente → aprovecha multinúcleo.

Java 7 – invokedynamic

- Nueva instrucción de bytecode JVM (JSR-292).
- Diseñada para soportar lenguajes dinámicos (JRuby, Groovy, Kotlin, Scala…).
- Cómo funciona:
  - Permite que las llamadas a métodos se resuelvan en tiempo de ejecución.
  - Usa call sites dinámicos y la API java.lang.invoke.

Era Oracle: Java 7 y Java 8 (II)

- Java 8 (2014) – Salto funcional y modernización
  - Expresiones lambda y API Streams (programación funcional, paralelizable).
  - Métodos por defecto en interfaces.
  - Nueva API de Fecha/Hora (JSR-310).
  - JavaFX integrado en el JDK.
  - Motor JavaScript Nashorn.
  - Referencias de método (Clase::metodo).
  - Versión LTS con soporte extendido, muy adoptada en entornos enterprise.

Java 8 – Expresiones Lambda

- Sintaxis compacta para interfaces funcionales (un solo método abstracto).
- Sustituyen clases anónimas → código más limpio.

```
Collections.sort(lista, new
Comparator<String>() {
   public int compare(String
s1, String s2) {
      return s1.compareTo(s2);
   }
});
```



```
Collections.sort(lista, (s1,
s2) -> s1.compareTo(s2));
```

Java 8 – API Streams

- Procesamiento declarativo de colecciones (filter, map, reduce).
- Ejecución secuencial o paralela (parallelStream()).

```
nombres.stream()
       .filter(n -> n.startsWith("P"))
       .map(String::toUpperCase)
       .forEach(System.out::println);
```

Métodos por defecto en interfaces

- Interfaces pueden tener implementación por defecto:

```
interface Animal {
   void hacerSonido();
   default void dormir() {
      System.out.println("Zzz...");
   }
}

class Perro implements Animal {
   public void hacerSonido() { System.out.println("Guau!"); }
}
```

Nueva API de Fecha/Hora (JSR-310)

- Date y Calendar → confusos, mutables, poco seguros.
- Con Java 8 → paquete java.time
  - LocalDate → solo fecha.
  - LocalTime → solo hora.
  - LocalDateTime → fecha + hora.
  - ZonedDateTime → con zona horaria.

```
LocalDate hoy = LocalDate.now();
LocalDate cumple = LocalDate.of(1999, 5, 20);

System.out.println("Hoy: " + hoy);
System.out.println("Días hasta cumple: " +
    ChronoUnit.DAYS.between(hoy, cumple));
```

JavaFX integrado en el JDK

- Sustituto moderno de Swing/AWT.
- Con Java 8 → JavaFX incluido en el JDK estándar.
- Características
  - Basado en Scene Graph.
  - Estilos con CSS y diseño con FXML.
  - Controles avanzados: tablas, gráficas, multimedia.
  - Aceleración por GPU.

Motor JavaScript Nashorn

- Sustituyó a Rhino (Java 6).
- Compatible con ECMAScript 5.1.
- Usa invokedynamic → mucho más rápido.

Referencias de método

- Forma más corta y legible de usar lambdas cuando lo único que se hace es llamar a un método existente.

```
lista.forEach(s -> System.out.println(s));
```



```
lista.forEach(System.out::println);
```

Nueva era de lanzamientos rápidos (Java 9 a Java 16)

- Modelo
  - Desde 2017: lanzamientos cada 6 meses.
  - Algunas versiones marcadas LTS (11, 17, 21…).No-LTS → soporte hasta la siguiente versión.
- Java 9 (2017)
  - JPMS / Proyecto Jigsaw (modularización del JDK).
  - jlink, colecciones inmutables (List.of), JShell REPL.
  - API HTTP/2 (incubador), fin de applets.
- Java 10 (2018)
  - var para inferencia de tipos locales.
  - JVM container-aware.

Java 9 – JPMS / Proyecto Jigsaw

- Sistema de módulos en Java.
- El JDK se dividió en ~90 módulos (java.base, java.sql, ...).
- Cada app puede definir sus módulos → más control y seguridad.

```
module mi.aplicacion {
    requires java.sql;
    exports com.ejemplo.servicio;
}
```

Java 9 – jlink, colecciones inmutables y JShell

- Jlink : Crea un runtime mínimo con solo los módulos necesarios.

```
jlink --add-modules java.base,java.sql --output miRuntime
```

- Colecciones inmutables

```
List<String> colores = List.of("rojo","verde","azul");
```

- JShell (REPL) : consola interactiva para probar código Java.

```
jshell> int x = 5; x+3
$1 ==> 8
```

Java 9 – API HTTP/2

- Sustituye a HttpURLConnection (Java 1.1).
- Incluida como incubadora (jdk.incubator.http).
- Soporta HTTP/1.1 y HTTP/2, asincronía con CompletableFuture.

```
HttpClient client = HttpClient.newHttpClient();
HttpRequest req = HttpRequest.newBuilder()
   .uri(URI.create("https://example.com"))
   .build();
HttpResponse<String> res = client.send(req,
   HttpResponse.BodyHandlers.ofString());
```

Java 9 – Fin de los Applets

- Applets = pequeños programas Java en navegadores (años 90–2000).
- Requerían plugin → problemas de seguridad y compatibilidad.
- En Java 9 quedaron deprecated oficialmente y eliminados completamente en java 11

Java 10 – var

- El compilador deduce el tipo de la variable a partir del inicializador.
- Solo para variables locales, bucles, try-with-resources.
- Tipado sigue siendo estático.

```
var n = 10;                 // int
var s = "hola";             // String
var lista = new ArrayList<String>();
for (var x : List.of(1,2,3)) { ... }
```

Java 10 – JVM container-aware

- JVM reconoce límites de CPU y memoria en contenedores (Docker/K8s).
- Ajusta heap y paralelismo automáticamente.
- Mejor integración con Docker/Kubernetes → heap y GC más predecibles.

Nueva era de lanzamientos rápidos (Java 9 a Java 16) (II)

- Java 11 (2018, LTS)
  - API HTTP Client final.
  - Eliminación Nashorn, JavaFX, Web Start y applets.
  - var en lambdas, nuevos métodos en String y Files.
  - Cambio de licenciamiento (Oracle JDK comercial).
- Java 12–13 (2019)
  - Switch Expressions (preview).
  - Shenandoah GC, Text Blocks (preview).
- Java 14 (2020)
  - Records (preview).
  - Pattern Matching instanceof (preview).
  - Switch Expressions finales.
  - ZGC experimental, NPE más descriptivos.

Nueva API HTTP (estándar en Java 11)

- En Java 9 apareció como incubador (jdk.incubator.http).
- En Java 11 se vuelve parte oficial: java.net.http.
- Soporta HTTP/1.1 y HTTP/2.
- API moderna, sencilla y fluida.
- Soporta llamadas asíncronas con CompletableFuture.

var en lambdas

- var ya existía en variables locales (Java 10).
- En Java 11 → también en parámetros de lambdas.

```
list.stream().map((var s) -> s.toUpperCase());
```

Mejoras en string – java 12

- String.indent(int n)
  - String.indent(int n)

```
String texto = "Hola\nMundo";
System.out.println(texto.indent(4))
;
```

- String.transform(Function< String, R> f)
  - Aplica una función al String y devuelve lo que produzca.

```
String resultado = "hola"
    .transform(s ->
s.toUpperCase())
    .transform(s -> ">> " + s + "
<<");
System.out.println(resultado);
```

Mejoras en manejo ficheros – java 12

- Files.mismatch(Path p1, Path p2)
  - Compara dos ficheros byte a byte.
  - Devuelve:
    - -1 si son idénticos,
    - El índice del primer byte diferente si no lo son.

```
long diff = Files.mismatch(Path.of("a.txt"), Path.of("b.txt"));
if (diff == -1)
    System.out.println("Son iguales");
else
    System.out.println("Difieren en el byte " + diff);
```

Java 12 – Switch Expressions (preview)

- Con Java 12 switch también puede devolver valores.
- Sintaxis compacta con ->.

```
int dia = 3;
String nombre = switch (dia) {
   case 1 -> "Lunes";
   case 2 -> "Martes";
   case 3 -> "Miércoles";
   default -> "Otro";
};
```

Java 12 – Shenandoah GC (experimental)

- Recolector de basura de baja latencia (RedHat).
- Minimiza las pausas del GC, incluso en heaps grandes.
- Compacción concurrente (a diferencia de G1/ZGC).
- Ideal para sistemas de tiempo real o finanzas.

Java 13 – Text Blocks (preview)

- Text Blocks con """ → cadenas multilínea limpias.

```
String json = """
{
  "nombre": "Ana",
  "edad": 30
}
""";
```

Java 14/16 – Pattern Matching con instanceof

- Antes:

```
if (obj instanceof String) {
    String s = (String) obj;    // cast manual
    System.out.println(s.toUpperCase());
}
```

- Ahora:

```
if (obj instanceof String s) {
    System.out.println(s.toUpperCase());
}
```

Records (preview en Java 14, estándar en 16)

- Crear clases "data carrier" (solo datos) implicaba mucho boilerplate (constructores, getters, equals, hashCode, toString).
- Con record → definición compacta e inmutable.

```
public record Persona(String nombre, int edad) {}

Persona p = new Persona("Ana", 30);
System.out.println(p.nombre()); // Ana
System.out.println(p); // Persona[nombre=Ana, edad=30]
```

Sealed Classes (preview en Java 15, estándar en 17)

- Permiten restringir qué clases pueden heredar de una clase o implementar una interfaz.
- Útil para modelar jerarquías cerradas.

```
public sealed class Forma
    permits Circulo, Rectangulo {}

final class Circulo extends Forma {}
final class Rectangulo extends Forma {}
```

ZGC (Z Garbage Collector, Java 15 estable)

- Recolector de basura diseñado para baja latencia y escalabilidad.
- Pausas del GC ≤ 10ms, incluso con heaps enormes (multi-TB).
- Todo el trabajo de recolección ocurre concurrentemente.

Nueva era de lanzamientos rápidos (Java 9 a Java 16) (III)

- Java 15 (2020)
  - Text Blocks finales.
  - Clases selladas (preview).
  - Foreign Memory API (incubador).
- Java 16 (2021)
  - Records y Pattern Matching finales.
  - Stream.toList().
  - Vector API (incubador).

Stream.toList()

- Antes:

```
List<String> lista =
   stream.collect(Collectors.toList());
```

- Con Java 16:

```
List<Integer> pares = List.of(1,2,3,4,5).stream()
   .filter(n -> n % 2 == 0)
   .toList();

System.out.println(pares); // [2, 4]
```

Herramientas y ejecución en la plataforma Java(I)

- JVM – Java Virtual Machine
  - Motor que ejecuta bytecode en cualquier SO.
  - HotSpot (desde JDK 1.3): compilación JIT adaptativa, GC generacional, soporte SIMD/cripto.
  - GCs: CMS, G1, ZGC, Shenandoah.
  - Modularidad desde Java 9 (class loaders modulares).
- JDK – Java Development Kit
  - Incluye compilador javac, ejecutor java, javadoc, jdb, jar, etc.
  - Contiene biblioteca estándar Java SE.
  - Desde 2007, basado en OpenJDK (GPL).
  - Distribuciones: Oracle, Temurin, Corretto, Red Hat, etc.
  - jlink (Java 9+): crear runtimes ligeros con módulos necesarios.

Herramientas y ejecución en la plataforma Java(II)

- JRE – Java Runtime Environment
  - JVM + bibliotecas estándar, sin herramientas de desarrollo.
  - Ya no se distribuye separado desde Java 9; se usan runtimes creados con jlink o JDK completo.
- Compiladores
  - javac (self-hosting desde Java 1.3).
  - Otros: ECJ (Eclipse), GCJ (obsoleto), GraalVM (JIT avanzado y compilación nativa AOT).
- Evolución de ejecución/despliegue
  - Años 90–2000: Applets (descontinuados en 2018–2019), Java Web Start (retirado en Java 11)
  - .Actualidad: JAR/WAR, fat JARs (Spring Boot), contenedores Docker/K8s (JVM container-aware).
  - jpackage: empaquetado nativo con runtime embebido.

Entorno empresarial (Java EE / Jakarta EE)

- Estándar para aplicaciones n-capas portables: servlets, EJB, JMS, JDBC.
- Servidores: WebSphere, WebLogic, JBoss/WildFly, Oracle AS.Uso masivo en banca, administración pública, comercio electrónico.Ventajas: independencia de proveedor, gran comunidad profesional.

Aplicaciones de escritorio

- Ejemplos: NetBeans, IntelliJ IDEA, Eclipse.
- Tecnologías: Swing, SWT, JavaFX.
- Uso en herramientas de administración y monitoreo multiplataforma.

Dispositivos móviles

- Java ME: estándar en teléfonos básicos (MIDP) y juegos móviles en 2000s.
- Android: sintaxis Java 5, VM Dalvik/ART, migración masiva de desarrolladores.

Sistemas embebidos e IoT

- Java Card: SIMs, tarjetas bancarias.
- Java TV / BD-J: decodificadores, Blu-ray.
- Presente en controladores, impresoras, pasarelas IoT.
- Raspberry Pi y dispositivos ARM corriendo Java SE reducido.

Ecosistema Java(I)

- IDEs
  - NetBeans (open source desde 2000), Eclipse (2001, extensible y gratuito), IntelliJ IDEA (2001, asistencia avanzada).
  - Funciones: refactorización, autocompletado, depuración, integración Maven/Gradle.
- Sistemas de construcción
  - Ant (2000): builds XML.
  - Maven (2004): dependencias + convención → estándar de facto.
  - Gradle (2009): DSL flexible (Groovy/Kotlin), muy usado en Android.

Ecosistema Java(II)

- Frameworks y librerías
  - Web: Struts (MVC), Spring (IoC, DI, Spring Boot), JSF, GWT, Vaadin.
  - Persistencia: Hibernate (ORM).
  - Servicios: JAX-RS, Jersey, RESTEasy, MicroProfile.
  - Utilidades: Apache Commons, SLF4J, JUnit, Big Data (Hadoop, Spark), lenguajes JVM (Kotlin, Scala).
- Comunidad y gobernanza
  - JCP (Java Community Process) + JSRs para evolución consensuada.
  - OpenJDK como implementación oficial (GPL).
  - Java EE transferido a la Fundación Eclipse → Jakarta EE.
  - Eventos: JavaOne, JavaLand, Java User Groups.

Licenciamiento y disputas legales

- De Sun a Oracle
  - Sun libera gran parte de Java (2006–2007, GPL).
  - Oracle adquiere Java (2010).
  - Cambio de licencia en Java 11 (uso comercial pago) → migración a OpenJDK.
  - Desde Java 17: uso gratuito también en producción.
- Caso Oracle vs Google (Android)
  - Android usó APIs Java SE sin licencia oficial (Dalvik/ART).
  - Disputa 2010–2021 → Corte Suprema de EE.UU. falla a favor de Google (uso justo).
  - Android ahora usa OpenJDK para APIs Java.

Evolución de versiones: de Java 8 a Java 17 y 21

- Nuevo modelo de lanzamientos (desde 2018)
  - 1 versión cada 6 meses.
  - LTS (Long Term Support) cada ~2 años → versiones estables para adopción empresarial.
  - LTS recientes: Java 8 (2014), Java 11 (2018), Java 17 (2021), Java 21 (2023).
  - Próxima LTS esperada: Java 25 (2025).

Java 17 (LTS – 2021)

- Records: clases inmutables y transparentes.
- Clases selladas (sealed): restringir herencia.
- GC de baja pausa: ZGC, Shenandoah.
- Modularización completa (desde Java 9).
- Eliminados Applets, Java Web Start, APIs Java EE y CORBA.

Java 21 (LTS – 2023)

- Hilos virtuales (Proyecto Loom) → millones de hilos sobre pocos hilos nativos → escalabilidad masiva.
- Nuevas interfaces: SequencedCollection, SequencedSet, SequencedMap.
- Métodos convenientes: getFirst(), getLast() en colecciones.
- Pattern matching para switch y records.
- Unicode 15 + soporte de emojis.
- API HTTP mejorada (cierre automático de conexiones).
- Preview: cadenas plantilla, concurrencia estructurada.

Hilos virtuales (Project Loom) – Java 21

- Threads ligeros gestionados por la JVM
- Permiten millones de hilos sobre pocos nativos
- Ideales para tareas de I/O concurrente (HTTP, BBDD, sockets)
- Modelo simple: “1 tarea = 1 hilo” sin coste elevado
- Integrados en Thread y ExecutorService

```
try (var exec = Executors.newVirtualThreadPerTaskExecutor()) {
    exec.submit(() -> System.out.println("Hola!"));
}
```

Nuevas interfaces en Java 21

- SequencedCollection, SequencedSet, SequencedMap
  - API unificada para colecciones ordenadas
  - Acceso coherente a primer / último elemento
  - Métodos clave ([getFirst(), getLast()], [addFirst(), addLast()], [removeFirst(), removeLast()], [reversed()])
- Implementaciones:
  - List, Deque → SequencedCollection
  - LinkedHashSet → SequencedSet
  - LinkedHashMap → SequencedMap

```
SequencedSet<String> s = new LinkedHashSet<>();
s.addFirst("A"); s.addLast("B");
System.out.println(s.reversed()); // [B, A]
```

Pattern Matching en Java 21

- switch + records
  - switch ahora entiende tipos y records
  - Se pueden desestructurar objetos directamente
  - Soporta guards (when) para condiciones extra
  - Evita instanceof + casting manual
  - Código más claro, seguro y expresivo

```
sealed interface Shape permits Circle, Rect {}
record Circle(double r) implements Shape {}
record Rect(double w, double h) implements Shape {}

double area(Shape s) = switch (s) {
  case Circle(var r)       -> Math.PI * r * r;
  case Rect(double w, double h) when w == h -> w * h;
  case Rect(double w, double h) -> w * h;
};
```

API HTTP mejorada – Java 21

- HttpClient (desde Java 11) con pool de conexiones
- En Java 21 → mejor cierre automático de conexiones
- Si no consumes el cuerpo → el cliente lo cierra/reutiliza
- Menos riesgo de fugas de conexión

```
var res = client.send(req, HttpResponse.BodyHandlers.ofInputStream());
try (var in = res.body()) {
    in.readAllBytes(); // conexión liberada
}
```

String Templates + Concurrencia estructurada

- String Templates (JEP 430)
  - Interpolación de cadenas más clara y segura
  - Procesadores (STR, FMT, personalizados)

```
String name = "Ada";
String msg = STR."Hola, \{name}!";
```

- Concurrencia estructurada (JEP 453)
  - Manejo de subtareas como unidad lógica
  - Si una falla → se cancelan todas

Modelo de licenciamiento: Oracle JDK vs OpenJDK

- Antes de Java 11 (hasta Java 8)
  - Oracle JDK gratuito para uso general (licencia Oracle Binary Code License).
- Cambio en Java 11 (2018)
  - Oracle JDK requiere suscripción comercial para producción.
  - Se impulsa el uso de OpenJDK (GPLv2).
  - Surgen múltiples distribuciones alternativas (AdoptOpenJDK, Azul Zulu, Amazon Corretto…).
- Ajustes posteriores (2021–2023)
  - Java 17 (2021): Licencia NFTC → uso gratuito hasta fin de soporte premier (ej. sept. 2024).
  - Tras periodo NFTC: se necesita licencia comercial (OTN).
  - 2023: nuevo modelo Java SE Universal Subscription → pago por total de empleados, no por instalaciones.

Ecosistema actual de JDKs

- OpenJDK gratuito siempre (código abierto, GPLv2).
- Builds alternativas con soporte prolongado:
  - Adoptium/Temurin (Eclipse Foundation).
  - Amazon Corretto (22% cuota empresarial).
  - Azul Zulu, IBM Semeru, Microsoft OpenJDK, etc.
- Compatibles entre sí (todas pasan el Java Compatibility Kit).

Mantenimiento y soporte en Java

- Nueva versión cada 6 meses.
- No LTS (Java 18, 19, 20…): soporte breve (~6 meses).
- LTS: soporte prolongado.
  - Oracle: ≥ 8 años (clientes suscritos).
  - Comunidad: ~4 años gratis (Adoptium, Red Hat…).
  - Ej.: Java 17 → parches comunitarios hasta 2027.

Funciones en preview

- Desde Java 9: activar en compilador (--enable-preview).
- Permite probar nuevas capacidades antes de ser finales.
- Ejemplos:
  - Records → preview en Java 14/15, final en 16.
  - Virtual Threads → preview en 19/20, final en 21.

Panorama Java en 2025

- Dos LTS recientes: Java 17 y Java 21.
- Flujo constante de mejoras en lenguaje y librerías.
- Ecosistema abierto con múltiples distribuidores del JDK.
- Oracle ya no es la opción por defecto → auge de OpenJDK y builds libres.

Distribución de Software Java

- Java permitió distribuir apps independientes del SO mediante la JVM.
- Antiguos métodos:
  - Applets en navegador (90s–2000s).
  - Java Web Start (JWS) para lanzar apps de escritorio vía web.
- Nuevas formas: microservicios en contenedores, ejecutables nativos, despliegue en la nube.

Applets y JWS en la actualidad

- Fin de los Applets
  - Ejecutaban código Java incrustado en páginas web vía plugin.
  - Deprecados en JDK 9 (2017), eliminados en JDK 11.
  - Navegadores eliminaron soporte NPAPI → incompatibles.
  - Sustituidos por HTML5/JS, apps de escritorio o soluciones específicas (ej. AutoFirma).
- Fin de Java Web Start
  - Permitía instalar/actualizar apps Java con enlaces .jnlp.
  - Deprecado en JDK 9, eliminado en JDK 11.
  - Alternativa: OpenWebStart (open source, compatible JNLP).

Java Modular (JPMS) y jlink

- Modularización (Java 9)
  - Introduce JPMS (Java Platform Module System).
  - Permite crear runtimes mínimos con solo los módulos necesarios.
  - Objetivo: reducir tamaño, mejorar seguridad y encapsulación.
- Jlink
  - Genera imagen de runtime personalizada:
    - JVM + librerías esenciales + clases de la app.
    - Tamaños reducidos (ej. 30 MB vs 100 MB de un JRE completo).
  - Ideal para contenedores Docker y entornos controlados.

jpackage (Java 14 → estable en Java 16)

- Empaqueta el runtime modular + app en instaladores nativos:
  - Windows: MSI / EXE
  - macOS: PKG / DMG
  - Linux: DEB / RPM
- Evita que el usuario tenga que instalar Java por separado.

Compilación Nativa con GraalVM

- Máquina virtual universal (Java, JavaScript, Python, R, C/C++ vía LLVM).
- Desarrollada por Oracle Labs.
- Puede ejecutar en modo JVM tradicional o generar binarios nativos (Native Image)
- Ventajas de la compilación nativa (AOT)
  - Arranque ultrarrápido: ms frente a segundos de la JVM.
  - Menor huella de memoria: decenas de MB en lugar de ~200 MB.
- Limitaciones
  - Pérdida de portabilidad: binarios por plataforma.
  - No soporta bien reflexión dinámica sin configuración.

Contenedores Docker en Java

- Docker en Java
  - Contenedor = app + runtime + dependencias → ejecución aislada y portable.
  - Ejemplo: amazoncorretto:17-alpine + app Spring Boot.
  - Ventajas: aislamiento, portabilidad, escalado, microservicios independientes.
- Buenas prácticas:
  - Imágenes base minimalistas (alpine, debian-slim).
  - JPMS/jlink para JRE reducido.
  - Construcción multi-fase (build → runtime).
  - Uso de herramientas como Jib o Buildpacks.

Kubernetes en Java

- Orquestación de contenedores a gran escala.
- Pods, ConfigMaps/Secrets, balanceo, escalado automático (HPA).
- JVM adaptada para detectar límites de CPU/RAM en contenedores (Java ≥10).
- Integración con métricas y logging estructurado (Micrometer, Prometheus).

De Java EE a Jakarta EE y MicroProfile

- Orígenes: Java EE
  - Estándar empresarial desde finales de los 90 (J2EE → Java EE).
  - APIs clave: Servlets, JSP/JSF, EJB, CDI, JPA, JDBC, JMS, JAX-WS.
  - Servidores compatibles: GlassFish, WebSphere, JBoss/WildFly, TomEE.
  - Desarrollo colaborativo bajo el Java Community Process (JCP).
- Transición a Jakarta EE (2018)
  - Oracle dona Java EE a Eclipse Foundation para acelerar la innovación.
  - Restricción legal: no usar el nombre “Java EE” → nuevo nombre Jakarta EE.
  - Inspirado en Apache Jakarta y la referencia al café Java.

De Java EE a Jakarta EE y MicroProfile(II)

- Principales hitos
  - Jakarta EE 8 (2019) → igual que Java EE 8, cambio solo de gobernanza.
  - Jakarta EE 9 (2020) → cambio de javax.* a jakarta.*.
  - Jakarta EE 10 (2022) → mejoras en Servlet 6.0, JSF 4.0, JAX-RS 3.1, Core Profile para microservicios.
  - Jakarta EE 11 (2024/25) → integración con Virtual Threads (Java 21), mejoras cloud-native y seguridad JWT/OAuth 2.1.

Impacto y panorama actual

- Jakarta EE hoy
  - Mismo núcleo que Java EE: APIs renombradas (jakarta.*), mismas funcionalidades clave.
  - Migración asistida por los principales servidores: GlassFish, WildFly, WebSphere Liberty, Payara, TomEE.
  - Gobernanza abierta en Eclipse Foundation → colaboración de Oracle, IBM, Red Hat, Payara, Tomitribe.
  - Adaptación a la nube: integración con Docker/Kubernetes, arquitecturas reactivas, seguridad moderna (JWT, OAuth).

Eclipse MicroProfile

- Optimiza Jakarta EE para microservicios.
- Incluye:
  - Configuración externa (MicroProfile Config).
  - Tolerancia a fallos (Fault Tolerance).
  - Métricas (Metrics).
  - Cliente REST tipado.
  - Seguridad JWT Propagation.
  - OpenAPI.
- Implementaciones: Payara Micro, WebSphere Liberty, WildFly (modo MicroProfile), Thorntail, KumuluzE

Dos caminos en Java empresarial

- Jakarta EE tradicional
  - Servidores certificados (GlassFish, WildFly, OpenLiberty).
  - Despliegue WAR/EAR.
  - Compatibilidad y estándares.
- Frameworks ligeros y cloud-native
  - Spring Boot, Quarkus, Micronaut, Helidon.
  - Servidores embebidos, arranque rápido, optimizados para contenedores y GraalVM.

Servidores de aplicaciones actuales

- Clásicos Jakarta EE
  - WildFly / JBoss (Red Hat)
    - Jakarta EE 10, soporte MicroProfile.
    - Modo modular y WildFly Bootable JAR.
  - WebLogic (Oracle)
    - Compatible Java EE 8 (javax) y Jakarta (jakarta) vía namespace switch.
  - WebSphere / Open Liberty (IBM)
    - Open source modular (Open Liberty) + versión comercial.
    - Jakarta EE + MicroProfile, apto para contenedores.
  - GlassFish / PayaraGlassFish
  - Apache Tomcat / Jetty
    - Contenedores Servlet (no full EE).
    - Muy usados embebidos en Spring Boot.
  - TomEE : Tomcat + APIs EE básicas (CDI, JAX-RS, JPA).

Alternativas ligeras y cloud-native

- Spring Boot → Tomcat/Jetty/Netty embebido.
- Quarkus (Red Hat) → optimizado para GraalVM, MicroProfile.
- Micronaut (Oracle) → compilación AOT, serverless-friendly.
- Helidon (Oracle)
  - SE: estilo handler funcional.
  - MP: implementación de MicroProfile.
- Akka → modelo de actores (nicho: telecom, finanzas).
- Serverless → Fn Project, AWS Lambda (Java con arranque optimizado).

¿Preguntas?